

Multi-Agent AI Support Ticket System

Group Project 1

Group 2: Parth Auti, Hasti Bhalodi, Medha Jani, Kento Hopkins

September 28, 2026

1. What It Does

You type in a computer problem, like "I forgot my password," and our system files a support ticket for it. It picks the category and writes the resolution notes itself.

There are two agents. The first one, the Requester, runs the whole job. It sends the problem to the second agent, waits for an answer, fills in the support form, and checks that the ticket came out right.

The second one, the Specialist, is the one that knows things. It searches our seven support documents, finds the parts that match the problem, and gives those to an LLM to write the answer. It sends back a category, some resolution notes, and which files it used.

The answer is not just printed on the screen. The category decides what gets picked in the dropdown, and the resolution notes are what goes in the resolution box. If the category does not match any option on the form, the Requester stops and does not submit.

2. How It Works

Here is the order things happen in:

1. The user types a problem and it goes into `handle_request()`.
2. The Requester sends it to the Specialist with `POST /tasks`.
3. The Specialist saves the task, starts a thread, and immediately sends back a task ID and the status `submitted`.
4. The Requester keeps calling `GET /tasks/{id}` to see if it is done.
5. The thread sets the task to `working` and runs the RAG pipeline.
6. The pipeline searches the documents and writes an answer.
7. The task becomes `completed` and the next check gets the answer.
8. The Requester turns the category into a dropdown value, and Playwright fills in the form and submits it.
9. The Requester reads the confirmation page and checks it says the right thing.

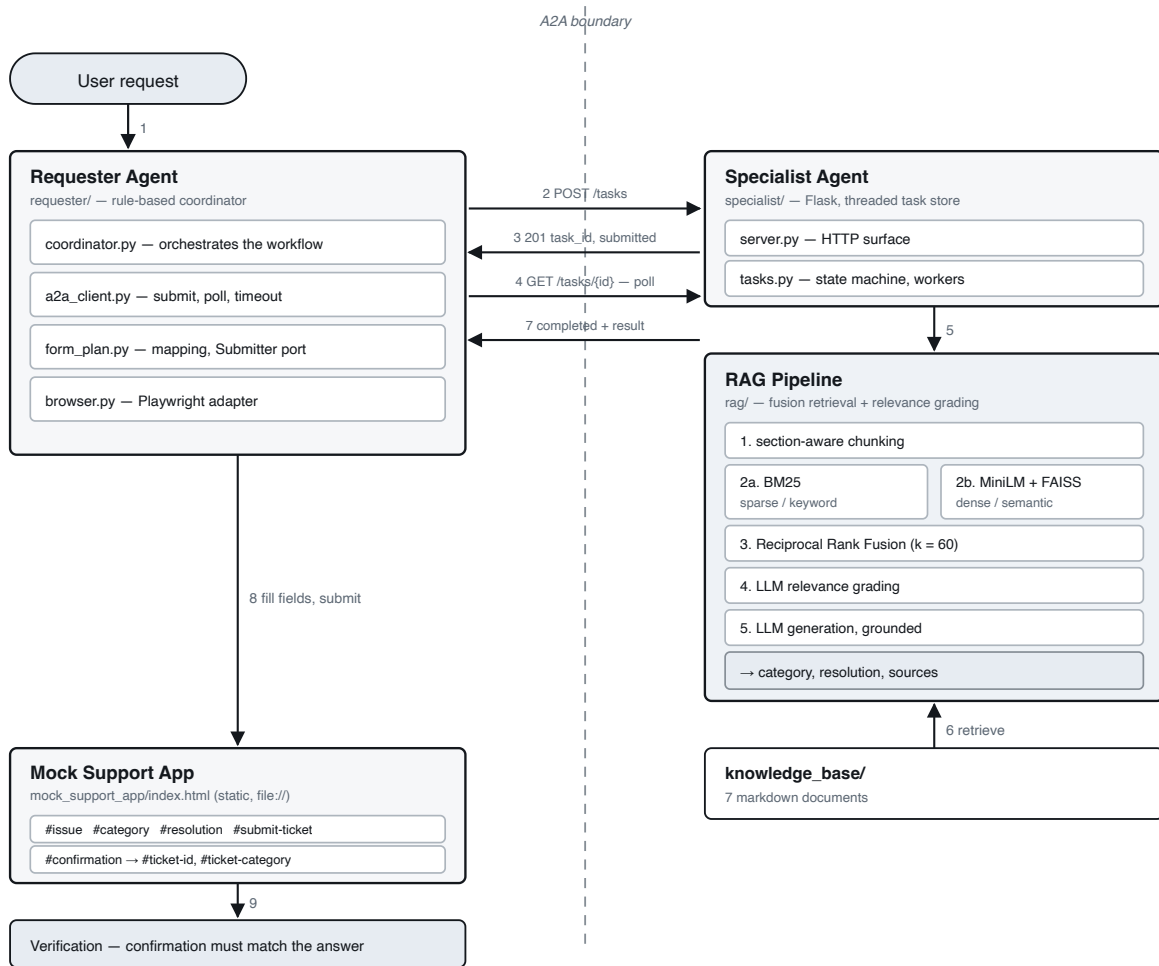


Figure 1: How the parts fit together. The dashed line is where the two agents split, since they run as separate programs and only talk over HTTP.

Why we split it up like this

We kept three rules while building it, and all three turned out to matter.

The Specialist does not know the web form exists. It just returns whatever category the documents use, and the Requester turns that into a dropdown value. The task code does not know about RAG either, because the pipeline gets handed in as a function. That one helped a lot, because we swapped out our whole search system halfway through and never had to touch the server. And the coordinator does not know about Playwright, because it goes through an interface instead. That meant the browser code could be written at the same time as everything else.

3. The A2A Protocol

Before writing any code we wrote the protocol down in a file and agreed nobody would change it without telling the group. That is probably why four people could work on four parts at the same time without breaking each other's stuff.

Endpoints

Endpoint	What it does
POST /tasks	Sends a task. Replies with a task ID and the status <code>submitted</code> . It does not wait for the work to finish.
GET /tasks/{id}	Gives the status. Includes the result once it is done, or an error if it failed. Returns 404 if the ID is wrong.
GET /health	Checks the server is running.

Task states

State	Done?	What it means
submitted	No	Accepted but not started.
working	No	The RAG pipeline is running.
completed	Yes	The answer is ready.
failed	Yes	Something went wrong. There is an error instead of an answer.

What the messages look like

[illegible]

```
"sources": ["password_reset.md"],  
"confidence": 0.82}}
```

Error codes

We gave each error a code instead of just a message, so the Requester can tell them apart.

`NO_RELEVANT_CONTEXT` means the documents had nothing useful, so no ticket should be filed.

`LLM_ERROR` means the model call itself broke. `UNKNOWN_TASK` and `BAD_REQUEST` mean the request was wrong. `TIMEOUT` and `SPECIALIST_UNREACHABLE` come from our side, not the server's.

Things we decided

The main one is that sending a task does not wait for the answer. The work happens on a background thread, so the reply comes back while the search is still running. If we had made it wait, the task IDs and the polling would not really be doing anything. We have a test that checks this by turning on a delay and making sure the reply still comes back fast.

We also had to make the first reply always say `submitted`. What was happening is the thread could sometimes change the task to `working` before the reply had even been sent, so you could submit a task and get back a status that did not match our own rules. Now the reply always says `submitted` no matter what the task has already changed to.

The last one gave us the most trouble. At first we wrapped the user's problem in extra instructions before sending it, something like "Using the support knowledge base, identify the correct category," and sent that whole thing as the question. But the Specialist uses the question as its search query, so all those extra words went into the search and buried the actual problem. Three of the five test cases stopped working. We moved the instructions into the RAG code where they belong, and now we just send the user's own words plus a separate field saying what we want back.

4. The RAG Pipeline

The pipeline has five steps.

1. **Split up the documents.** Each one gets split by its markdown headings. We only keep the sections that describe problems, which are Description, Symptoms, and When to Use.
2. **Build two search indexes.** One is BM25, which is keyword based. The other uses `all-MiniLM-L6-v2` embeddings stored in FAISS, which is meaning based. Both cover the same chunks. We save the index to disk so it only gets rebuilt when the documents change.
3. **Combine the two.** Each index gives its own ranked list, and we merge them using Reciprocal Rank Fusion with $k = 60$. We keep the top eight.

4. **Grade them.** We ask the LLM in one call whether each of those eight passages is actually relevant, and throw out the ones that are not. If nothing is left, the task fails with `NO_RELEVANT_CONTEXT`.
5. **Write the answer.** The category is copied straight from the document's own Ticket Category section. The LLM writes one to three sentences of resolution notes using only that document's Resolution text.

The advanced technique we used

We used fusion retrieval, which means running keyword search and meaning based search separately and then combining the two rankings, plus grading the results with the LLM before writing anything. Neither of those is in the basic group of techniques.

We picked it because of what our documents are like. All seven use the same template, with the same headings and a lot of the same sentences. "Restart the computer if appropriate" is in four of them. Embeddings pick up on all that repeated structure, which makes the documents look more similar than they really are. If `network.md` gets confused with `hardware.md`, the wrong option gets picked in the dropdown. BM25 keeps the exact words that tell them apart, like "Wi-Fi" and "keyboard". Using both together keeps that while still being able to connect something like "cannot log in" to "account locked."

We added the grading step for a different reason. Without it, if you ask something the documents do not cover, the search still returns the closest thing it has and the LLM will write a confident answer from it anyway. Grading lets the system say that nothing here actually answers this, which is what makes our `NO_RELEVANT_CONTEXT` case real instead of something we just say exists.

Smaller choices

- We only index the problem sections. When we included the Resolution sections too, every document started looking the same, because the resolutions are all written the same way.
- The category is copied from the document instead of generated. It has to exactly match one of four options, so there is no reason to let a model reword it.
- Grading is one call with all eight passages numbered, instead of eight separate calls, to make it faster and cheaper.
- The file names always come back with the answer, so you can check where it came from.

5. The Playwright Part

The mock app is just an HTML file with no backend, so we open it with a `file://` URL. Then the code does this:

1. Starts Chromium and opens the page.
2. Types the user's problem into `#issue`.
3. Picks `#category` using the value we worked out from the category.
4. Types the resolution notes into `#resolution`.
5. Clicks `#submit-ticket`.
6. Waits for `#confirmation` to appear. If it never does, we read whatever is in `#error` on the page and fail with that message.
7. Reads the ticket ID, category, and resolution off the confirmation, and checks the ID is five digits.
8. Closes the browser in a `finally` block so it always shuts.

Checking it actually worked

A confirmation box showing up only proves the form took something. So we also compare the category and resolution on the confirmation page against what the Specialist sent us, and fail if they are different. That way a ticket filed under the wrong category cannot count as a success.

Matching categories to the dropdown

The Specialist returns categories the way the documents write them, so we have to match those to the four values the dropdown accepts:

Category in the documents	Option on the form
Account Access	account_access
Hardware	hardware
Software	software
Network	network
Email	not on the form
Security	not on the form

This is where we ran into something we did not expect. There are documents for Email and Security, and one of the test cases is an email problem that expects the category "Email," but the dropdown has no

Email option. Instead of just picking whichever one is closest, we made it raise `UNMAPPABLE_CATEGORY` and refuse to submit. We check this before the browser even opens, so it fails fast. Our thinking was that filing a ticket under a category we already know is wrong is worse than not filing one.

6. Error Handling

Error	When it happens	What the system does
<code>NO_RELEVANT_CONTEXT</code>	Nothing found in the documents was relevant.	The task fails. We report it and never open a browser.
<code>TIMEOUT</code>	The task was still going when we gave up waiting.	Stops checking and says how long it waited and how many times it checked.
<code>UNMAPPABLE_CATEGORY</code>	The category is not on the form.	Refuses before opening the browser and says which category it got.
<code>LLM_ERROR</code>	The model call broke or gave back something we could not read.	Any crash in the worker becomes a failed task instead of killing the server.
<code>VERIFICATION_FAILED</code>	The confirmation never showed up, or did not match.	Fails and includes whatever error the page showed.
<code>SPECIALIST_UNREACHABLE</code>	The Specialist is not running.	Says so right away instead of hanging.
<code>UNKNOWN_TASK</code>	The task ID does not exist.	Returns 404 and we stop checking.

Two things made this easier. The coordinator never throws an exception. Every failure comes back as a normal return value with an error code on it, so we can show any of these cases on demand. And any crash inside a worker gets caught and turned into a failed task, so a bug in the search only breaks one ticket instead of the whole agent.

7. Results

We measured two things. First the search on its own, then the whole system.

Search results

For each of the five test cases we checked which document each search method put first, and whether that document has the right category on it. There is no LLM in this test, which we did on purpose so the numbers only show the search change and nothing else.

Search method	Right document first
Basic keyword matching	4 out of 5
Fusion retrieval (BM25 + embeddings + RRF)	5 out of 5

Test case	Should be	Basic picked	Fusion picked
Forgot password, cannot log in	Account Access	account_access.md	password_reset.md
Laptop will not connect to Wi-Fi	Network	hardware.md (wrong)	network.md
Keyboard stopped working	Hardware	hardware.md	hardware.md
Cannot install software	Software	software.md	software.md
Email not syncing	Email	email.md	email.md

The one that changed is the one we were hoping would. For "My laptop won't connect to Wi-Fi," the basic version picks `hardware.md`, because the word "laptop" is all over the hardware symptoms and beats the one mention of "Wi-Fi." Fusion retrieval picks `network.md` instead, because the BM25 half keeps the word that actually separates the two documents.

The password one is interesting too. Both get the right category, but fusion picks `password_reset.md` where the basic one picks `account_access.md`. Both files say the category is Account Access so the ticket is the same either way, but the more specific document is the better match.

The whole system

Running all five test cases through everything, meaning both agents, the protocol, and Playwright opening a real Chromium window, gives four tickets submitted with valid five digit IDs and the right categories. The fifth one stops at `UNMAPPABLE_CATEGORY` because it is an email problem and the form has no Email option. We are counting that as working correctly, since the only other choice was filing it under a category we knew was wrong.

8. Reflection

What went well

Writing the protocol down before we started coding was the best thing we did. It meant all four of us could work at once, each testing against a fake version of everyone else's part, and it mostly just fit together when we put it together. The two places where you can swap something in, where the RAG pipeline goes in as a function and the browser sits behind an interface, are what made that actually work.

We replaced our entire search system partway through and did not have to change anything in the agent that calls it.

Having the coordinator return a result object instead of throwing exceptions also helped more than we thought it would. Every failure became something we could trigger and show instead of just describe.

What did not go well

Our first version put the prompt wording into the message between the agents, which quietly broke three of the five test cases because those words ended up in the search. It took us a while to figure out, because nothing crashed. The answers just got worse. What we learned is that the search query and the prompt you give the model are two different things and should not be in the same place.

We also had a state in our protocol that existed but could never be seen. Our fake delay was in the wrong spot, before the switch to `working` instead of after it, so tasks went from `submitted` straight to `completed` and you could never catch the middle state. The code looked fine and was still wrong. That is why we wrote tests that check each state can actually be seen, not just that it exists in the code.

How well the RAG worked

The difference between the two search methods came out to one case out of five. That is good but we do not want to make too much of it, because five test cases is not enough to really tune anything, and one result changing could just be luck. What it does show is that the problem we guessed at from reading the documents is the same one fusion retrieval fixed.

The bigger thing we noticed is that how well search works on these documents depends way more on how similar they are to each other than on how much text there is. With seven documents all written from the same template, the hard part is telling them apart, not finding them. That is why mixing the two search types beat either one on its own, and why picking which sections to index mattered more than picking a chunk size. A much bigger set of documents would probably be different.

Limitations

- Tasks are only kept in memory, so restarting the server loses them all.
- There are no retries, so one bad LLM call fails a ticket that would have been fine.
- You cannot cancel a task, so if we time out the work keeps going and finishes into an answer nobody reads.
- The confidence number is just a search score. It is not a real probability.
- The Requester only knows about one Specialist and has no way to find others.

What we would do next

- Add retries, and tell the difference between a temporary problem and a task that really cannot be done.
- Save tasks somewhere so they survive a restart, and add cancelling.
- Have the Specialist describe itself somewhere so the Requester could find agents and pick between them. We would need that before adding a second specialist.
- Build a bigger test set, since five cases is not enough to trust.
- Figure out what should actually happen with Email and Security, either by adding them to the form or by sending them somewhere else, instead of just refusing.

9. Who Did What

Name	Work
Parth Auti	The Requester and Specialist agents (steps 1 and 2), the contracts file, and setting up the project.
Hasti Bhalodi	The A2A protocol (step 3): task states, the error codes, and the protocol tests.
Medha Jani	The RAG pipeline and advanced technique (step 4): splitting documents, both indexes, RRF, grading, and generation.
Kento Hopkins	The Playwright automation and checking (steps 5 and 6), and the browser side error handling.

Appendix: How to Run It

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
playwright install chromium
cp .env.example .env          # add GROQ_API_KEY for the full pipeline

# terminal 1
python -m specialist.server --delay 1

# terminal 2
python -m requester.cli --all --browser
python -m unittest test_a2a_protocol -v
```

To see the timeout, start the server with `--delay 999` and run the Requester with `--timeout 3`. To see `NO_RELEVANT_CONTEXT`, ask it something the documents do not cover, like "What is the capital of France?"